TECHNICAL REPORT · CAPSTONE PROJECT

JCL Analyzer & Abend Assistant

A citation-grounded retrieval-augmented assistant for z/OS batch job analysis and abend diagnosis

SPONSOR	Apex Systems — Use Case 2
PREPARED BY	Insper and Missouri S&T capstone teams
REPOSITORY	`capstone-insper/202602_APEX_SYSTEMS_JCL_Assistant`
REVISION	`main @ 4dd40ca`
DATE	November 5, 2026

This report describes the system as implemented in the repository at the revision above, including its first end-to-end evaluation against the deployed service (November 4, 2026).

ABSTRACT

Mainframe batch failures are diagnosed today by engineers who read job logs, look up IBM abend codes and system messages across thousands of pages of manuals, and correlate them with the Job Control Language (JCL) of the failing job. This knowledge is scarce and concentrated in a shrinking workforce. We present the *JCL Analyzer & Abend Assistant*, a web-based chat assistant that explains the JCL of the AWS CardDemo reference application and diagnoses IBM z/OS abend codes and messages. The system combines deterministic, regex-based identification of codes, messages, jobs and steps with exact dictionary lookup over chunked IBM manuals, dense semantic retrieval over a Chroma vector index, and a tool-using large language model (Claude Sonnet 4.5 on Amazon Bedrock) constrained to answer only from numbered sources and to cite every factual sentence. A post-processing stage renumbers and validates citations so that every marker in an answer maps to a verifiable source. The system is deployed as a FastAPI service behind nginx on a single EC2 instance, with a React front end and session-based authentication. In its first evaluation against the deployed service, all ten scenarios — covering job explanation, four system abends, three system messages and an out-of-scope refusal — passed both source and content checks, with latencies of 2–11 s and an approximate cost of US$0.04 per answer.

Contents

1 Introduction and Motivation

2 System Overview

3 Architecture

4 Methods

- 4.1 Corpus and Ingestion
- 4.2 Embedding and Indexing
- 4.3 Deterministic Code and Job Recognition
- 4.4 Grounded Generation with Tool Use
- 4.5 Citation Integrity
- 4.6 Conversation Context
- 4.7 Authentication and Abuse Controls

5 Deployment

6 Usage

7 Testing and Evaluation

8 Limitations and Future Work

9 Conclusion

References

A Appendix: Configuration Reference

1 Introduction and Motivation

A large share of transaction processing in banking, insurance and government still runs as batch jobs on IBM z/OS, described in Job Control Language. When a job step fails, the operating system terminates it with a *system completion code* (an "abend", such as `S0C7`, a data exception) and writes system messages to the job log (such as `IEF450I`, which reports the abend, or `IEF212I`, a data set not found). Diagnosing the failure requires three pieces of knowledge at once: what the code or message means, as documented in IBM's *MVS System Codes* and *MVS System Messages* manuals; how the failing job is set up — which program the step runs and which data sets its DD statements allocate; and the usual corrective action.

This knowledge is held by a shrinking population of experienced mainframe engineers, and the reference material is large: the two IBM manuals used here alone run to well over a thousand pages. Newer engineers and modernization teams therefore spend significant time on lookups that are mechanical but error-prone; codes that differ by one character (`S0C4` protection exception versus `S0C7` data exception) have unrelated causes.

General-purpose large language models can explain these codes fluently, but in an operational setting fluency without provenance is a liability: a plausible but wrong explanation, or an invented JCL parameter value, can send an engineer in the wrong direction during an incident. The design goal of this project is therefore not merely to answer, but to answer **only from authoritative sources, with a citation on every factual claim, and to refuse explicitly** ("Not found in the docs.") when the sources do not support an answer.

1.1 Scope

The proof of concept targets two corpora: (i) the JCL and catalogued procedures of **CardDemo**, an open-source mainframe credit-card application published by AWS as a modernization reference [1], and (ii) IBM's public **z/OS 3.1 MVS System Codes** [2] and **MVS System Messages, Volume 8 (IEF–IGD)** [3]. Questions about other topics (for example, REXX programming) are deliberately out of scope and should be refused.

2 System Overview

Users interact through a chat interface. A user may ask about a job ("What does the POSTTRAN job do?"), paste a fragment of a job log ("`IEF450I POSTTRAN STEP15 - ABEND=S0C7`"), or ask a free-form diagnostic question. Each answer is a short, plain-English explanation — cause first, then fix — in which each sentence carries bracketed markers such as `[1]` that link to the cited IBM entry (book, code, page) or CardDemo JCL member (file, line range). The interface also surfaces detected codes, job and step as chips, and shows source snippets in a side panel.

All interaction flows through a single HTTP contract:

```
POST /ask
{"question": "STEP15 ended S0C7, why?",
 "history":  [{"question": "...", "answer": "..."}]}   // optional

200 OK
{"answer": "... invalid packed decimal data [1] ... [2]",
 "codes":  ["S0C7"],
 "citations": [
   {"source": "z/OS MVS System Codes",
    "locator": "0Cx, p. 194", "snippet": "..."},
   {"source": "carddemo/app/jcl/POSTTRAN.jcl",
    "locator": "lines 18-45", "snippet": "..."}]}
```

Citation n in the answer text corresponds to element $n-1$ of `citations`. The endpoint requires an authenticated session (401 otherwise), returns 502 when the model is unavailable and 503 when the corpus or index has not been built.

3 Architecture

The system is a three-tier application organized as a monorepo (Table 1). The Python back end imports the retrieval-and-generation package directly rather than calling it over the network, which keeps the deployment to a single process.

Component	Technology	Responsibility
`frontend/`	React, Vite	Sign-in screen; chat with per-user history in browser storage; clickable citations and sources panel; live detection of codes, job and step as the user types.
`backend/`	FastAPI, SQLite, Pydantic	`POST /ask`; `/auth/login`, `/auth/logout`, `/auth/me`; request validation; error mapping; account administration CLI (`python -m api.users`).
`rag-llm/ingest`	pypdf, git, ChromaDB	Fetch and chunk CardDemo JCL and the IBM manuals; build the vector index.
`rag-llm/rag`	ChromaDB, boto3 (Bedrock Converse)	Exact code lookup, job/step lookup, semantic search, tool-using generation, citation post-processing (`answer()`).
`infra/`	Bash, systemd, nginx	Idempotent install/update script for Amazon Linux 2023 on EC2.
`eval/`	Python, YAML	Ten evaluation scenarios and a scorer that exercises the deployed `/ask`.

Table 1. Repository components.

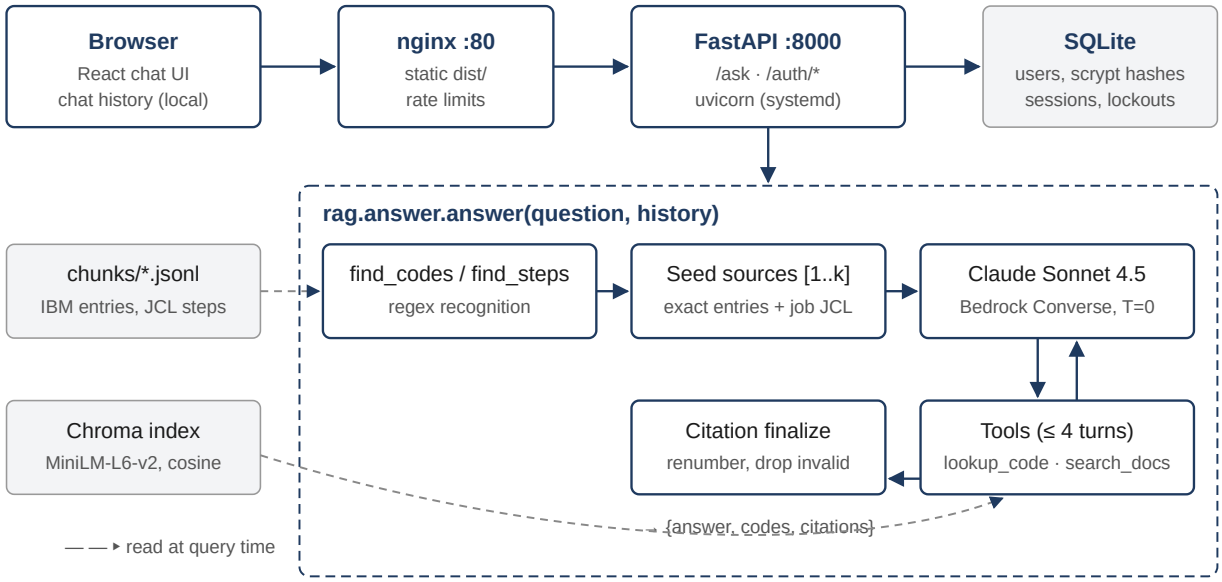


Figure 1. Request path. nginx serves the built front end and proxies `/ask`, `/auth` and `/docs` to FastAPI, which authenticates the session and calls the RAG pipeline in-process.

4 Methods

4.1 Corpus and Ingestion

CardDemo JCL (`ingest/jcl.py`). The CardDemo repository is sparse-cloned at a pinned commit (`59cc6c2`) so the corpus is reproducible, restricted to six directories of JCL and PROC members. Each member is split *structurally* rather than by fixed length: one chunk for the job card (everything before the first step) and one chunk per step, beginning at its `EXEC` statement and extended upward over the comment lines that immediately precede it, since those comments usually describe the step. Columns 73–80 (the JCL sequence field, ignored by the system) are stripped, and the Apache licence banner is removed as retrieval noise. Each chunk records its job name, step name, the program or procedure executed (parsed from `PGM=` / `PROC=`), and its 1-based start and end line, which later become the citation locator.

IBM manuals (`ingest/ibm.py`). Text is extracted page by page with pypdf and cached, since extraction takes roughly 30 s per book. Page footers are recognised by pattern and removed; the *printed* page number they contain is retained so that citations match the page numbers a reader sees in the manual, while the PDF page index is kept to build a deep link (`…pdf#page=N`). The System Codes book is restricted to the chapter on system completion codes. Entries are segmented at headings that match the code or message-identifier pattern (`0C7`, `0Cx`, `IEF450I`), with a validation rule that eliminates false headings: a candidate is accepted only if an `Explanation` section follows before any other section label or heading. This rejects module names and cross-references that happen to look like identifiers. The result is one chunk per IBM entry, keyed by its code.

4.2 Embedding and Indexing

Chunks from both corpora are embedded into a single Chroma collection using cosine distance and Chroma's default local embedding model, all-MiniLM-L6-v2 [5], chosen because it is free, runs offline after a one-time ~80 MB download, and keeps the deployment independent of a paid embedding API. Because this model reads only about 256 tokens, long entries are split at line boundaries into parts of at most ~700 characters, and *each part is prefixed with a header* repeating its identity — e.g. "JCL job POSTTRAN step STEP15 runs CBTRN02C (carddemo/app/jcl/POSTTRAN.jcl, lines 18-45)" or "0Cx: z/OS MVS System Codes, 0Cx, p. 194" — so that every embedded part carries its context. Metadata on each part records the parent chunk, its kind (`jcl_step`, `jcl_job`, `ibm_code`, `ibm_message`), source, locator and URL.

At query time (`rag/search.py`) the search may be filtered to the JCL or IBM corpus; the IBM side is roughly fifteen times larger and would otherwise crowd out JCL results. Because a single long entry (for example, `EC6`) may be split into dozens of parts, the search over-fetches by a factor of twenty and then collapses parts to one hit per parent chunk, keeping the best-scoring part.

4.3 Deterministic Code and Job Recognition

Semantic similarity is a poor tool for distinguishing `S0C4` from `S0C7`: the two IBM entries are textually near-identical in structure. The system therefore identifies codes deterministically before any retrieval (`rag/codes.py`). Regular expressions recognise system abends with optional reason codes (`S806-04`), the common typing error of a letter O for zero (`SOC7`), bare codes introduced by "abend" or "completion code" (`ABEND=S322`, "abend 0C4") while rejecting phrases such as "return code 100", and message identifiers in the IEF/IEC/IGD families. Codes are normalised and de-duplicated in order of appearance.

Lookup is an exact dictionary match over the IBM chunks, built in memory from the same JSONL file as the index so there is no second table to keep synchronised. IBM groups some codes into family entries (all of `0C1`–`0CF` are documented under `0Cx`); the lookup falls back to the family entry, and a focusing routine extracts the window of that entry around the requested sub-code, so the model receives the `0C7` paragraph rather than the first 6,000 characters of the family.

Analogously, `rag/jobs.py` recognises CardDemo job names in the question and returns that job's step chunks — only the named step if the question names one. At most two jobs are matched, because some job names coincide with DD names. The same recognition rules are mirrored in the front end (`src/detect.js`), which additionally parses job and step from `IEFnnnI` log lines to display context chips as the user types.

4.4 Grounded Generation with Tool Use

Generation (`rag/answer.py`) uses Claude Sonnet 4.5 [4] through the Amazon Bedrock Converse API at temperature 0 with a 1,500-token output limit. The pipeline proceeds as follows:

1. **Seeding.** The exact IBM entries for every recognised code and the JCL of every recognised job are placed in the first user message as numbered sources. Codes with no IBM entry are listed explicitly.

Seeding saves a model round-trip in the common case and guarantees that the model sees the correct entry rather than a semantically similar one.

2. **Tool loop.** The model may call two tools: `lookup_code(code)` for an exact entry, and `search_docs(query, corpus)` for semantic search over the `jcl` or `ibm` corpus. Each tool result is appended to the same numbered source list. The loop is capped at four model calls per question to bound latency and cost; exhausting the cap yields the refusal string.
3. **Constraints.** The system prompt requires the model to use only the numbered sources; to give no parameter value, example setting or number absent from a source; never to write example or corrected JCL; to cite every factual sentence; and to reply exactly "Not found in the docs." when sources are insufficient. When a job is named, the answer must connect the IBM explanation to that job's program and DD statements, but must not claim a specific JCL line caused a failure unless the question reports that error — JCL shows configuration, not what happened in a given run.

> The design choice is to let deterministic code do what it does reliably (identify *which* entry is relevant) and reserve the model for what it does well (reading several entries and the job's JCL and composing a short causal explanation), while making every claim auditable.

4.5 Citation Integrity

A `_Sources` registry assigns numbers to sources as they are shown to the model and de-duplicates by (source, locator): if a tool returns an entry already shown, the model is given the existing number and, unless the text is a new excerpt, the note "(already given above)". After generation, `finalize()` scans the answer for `[n]` markers, keeps only those that reference a real source, renumbers them 1..*k* in order of first use, removes markers that point at nothing, and emits exactly the *k* cited sources in that order. Sources shown to the model but not cited are not returned. The guarantee to the user is that every marker in the answer resolves to a source with a snippet, and that the citation list contains nothing the answer does not use.

4.6 Conversation Context

The client sends earlier turns with each request. The last three turns are replayed to the model as user/assistant message pairs, each truncated to 2,000 characters. Citation markers are stripped from earlier answers because numbering restarts every turn; an old `[1]` would otherwise point at the current turn's first source. Empty turns are dropped, as Bedrock rejects empty text blocks. Request validation bounds questions to 4,000 characters and history to ten turns.

4.7 Authentication and Abuse Controls

Because every `/ask` call incurs a model charge, the API is not public. Accounts are created by an administrator through a command-line tool; there is no self-registration. Passwords are hashed with scrypt at the OWASP minimum parameters ($N = 2^{15}$, $r = 8$, $p = 3$; about 32 MiB per hash) using only the Python standard library, and stored in SQLite. A bounded semaphore limits concurrent hash computations to four so that a burst of sign-ins cannot exhaust the memory the vector index also requires.

A successful sign-in issues a random session token in an HttpOnly, SameSite=Strict cookie valid for 12 hours, or 30 days with "Keep me signed in"; any token the browser already held is revoked first. Five failed attempts lock an email for 15 minutes, counted atomically so parallel guesses cannot exceed the limit, and unknown emails are locked identically so responses do not reveal whether an account exists. Validation errors are re-shaped so that the default FastAPI 422 response does not echo submitted passwords. nginx adds per-IP rate limits on sign-in and on `/ask`.

5 Deployment

The reference deployment is a single Amazon Linux 2023 EC2 instance (t3.medium) whose instance role grants access to Claude Sonnet 4.5 on Bedrock and read access to a private S3 prefix; no credential is stored in the code or the repository. Because ibm.com refuses scripted downloads, the two IBM PDFs are downloaded once in a browser and uploaded to that prefix. `infra/setup.sh`, run from AWS Session Manager, is idempotent and performs:

1. Installation of git, Python 3.11, nginx and Node 22; creation of an unprivileged `jcl` system user with a mode-700 data directory for the account database.
2. Clone or fast-forward of the repository; creation of a virtual environment and dependency installation.
3. Retrieval of the IBM PDFs from S3 and, *only when the ingest code has changed*, a rebuild of the corpus and index. Change is detected by comparing the git tree hash of `rag-llm/ingest` with a stamp file written after the last complete build; an interrupted build leaves no stamp and is therefore redone.
4. Build of the front end and installation of a systemd unit running uvicorn on the loopback interface, with `UMask=0077` so the database is readable only by the service user.
5. An nginx configuration serving `frontend/dist` on port 80 and proxying `/ask`, `/auth` and `/docs`.

6 Usage

6.1 Local Development

A single virtual environment at the repository root serves both Python packages. The stub mode (`JCL_ASSISTANT_STUB=1`) returns a fixed answer and requires neither AWS credentials nor a built index, so the user interface and API can be developed independently of the model.

```
python -m venv .venv && source .venv/bin/activate
pip install -r backend/requirements.txt          # includes rag-llm/requirements.txt

cd rag-llm
python -m ingest.jcl      # clone CardDemo, chunk JCL         ->
data/chunks/jcl.jsonl
python -m ingest.ibm      # chunk IBM PDFs in data/raw/ibm     ->
data/chunks/ibm.jsonl
python -m ingest.index    # embed and index (~4 min)           -> data/index

cd ../backend
python -m api.users add you@company.com --name "Your Name"
PYTHONPATH=../rag-llm JCL_ASSISTANT_STUB=1 uvicorn api.main:app --reload

cd ../frontend && npm ci && npm run dev   # http://localhost:5173, proxies to :8000
```

6.2 Command-Line Query

With AWS credentials available, the pipeline can be exercised directly without the web tier:

```
cd rag-llm
python -m rag.answer "STEP15 in POSTTRAN ended with S0C7, why?"
```

6.3 Account Administration

```
python -m api.users add alex@company.com --name "Alex Rivera"   # prompts for
password
python -m api.users list
python -m api.users reset-password alex@company.com            # also ends sessions
python -m api.users disable alex@company.com                   # and: enable
python -m api.users unlock alex@company.com                    # lift a lockout
```

6.4 Typical Interactions

Supported question forms include job explanation ("What does CLOSEFIL do and why would I run it before the batch?"), pasted log lines ("`IEF212I POSTTRAN STEP15 DALYTRAN - DATA SET NOT FOUND`"), abend diagnosis with job context ("CREASTMT abended S322 in STEP040. How do I fix it?"), JCL errors ("the log says `IEF453I CREASTMT - JOB FAILED - JCL ERROR`"), and follow-ups that rely on conversation history ("what does line 41 do?").

7 Testing and Evaluation

7.1 Automated Tests

Unit tests cover each layer without AWS access or a built corpus: JCL and IBM chunking, index construction, code recognition and lookup, job/step lookup and the generation loop with a fake Bedrock client in `rag-llm`; the `/ask` contract, sign-in, sessions, lockout and the accounts CLI in `backend`; the evaluation scorer in `eval`; and API client, chat storage, detection and formatting in the front end. GitHub

Actions runs all suites, plus front-end lint and production build, on every pull request and on every push to `main`.

7.2 Evaluation Methodology

`eval/scenarios.yaml` defines ten scenarios drawn from real CardDemo jobs. Each specifies a question, an expected cause written by the team, keyword groups and the sources that must be cited. `eval/run_eval.py` signs in and submits each question to `/ask` exactly as the chat client does, and scores two binary criteria:

- **source_ok** — every expected source appears among the returned citations (for the out-of-scope scenario: no citations are returned);
- **answer_ok** — the answer contains at least one term from every keyword group and no forbidden term (for the out-of-scope scenario: the answer begins with the refusal string).

Results, including the full answer and the expected cause, are written to CSV for human review alongside the automatic scores.

7.3 Results

The first evaluation against the deployed EC2 service was run on November 4, 2026 (`eval/results/2026-11-04-1319.csv`). Table 2 summarises the outcome.

Scenario	Codes detected	Citations returned	Src	Ans	s
`jcl-posttran`	—	POSTTRAN.jcl 18–45	✓	✓	4.8
`jcl-intcalc`	—	INTCALC.jcl 18–44	✓	✓	6.0
`jcl-closefil`	—	CLOSEFIL.jcl 18–34	✓	✓	5.9
`abend-s0c7-posttran`	IEF450I, S0C7	Messages IEF450I p.257; Codes 0Cx p.194; POSTTRAN.jcl	✓	✓	10.8
`abend-s0c4-intcalc`	S0C4	Codes 0Cx p.194; INTCALC.jcl	✓	✓	10.2
`abend-s806`	IEF450I, S806	Codes 806 p.497; INTCALC.jcl	✓	✓	6.3
`abend-s322-creastmt`	S322	Codes 322 p.371; CREASTMT.JCL 76–97	✓	✓	6.2
`msg-ief212i-dalytran`	IEF212I	Messages IEF212I p.134; POSTTRAN.jcl	✓	✓	7.0
`jcl-error-ief453i`	IEF453I	Messages IEF453I p.260; CREASTMT.JCL	✓	✓	8.5
`not-found-rexx`	—	(none — refused, as expected)	✓	✓	2.1

Table 2. First evaluation against the deployed service: 10/10 source checks and 10/10 answer checks passed. Latency is end-to-end `/ask` time in seconds.

All ten scenarios passed both criteria. Answers that required only job JCL completed in about 5–6 s; abend diagnoses combining IBM entries with job JCL took 6–11 s; the out-of-scope refusal took 2.1 s.

The approximate cost per answer was US$0.04. Two observations are worth noting. In the S0C7 and S0C4 scenarios the correct sub-code paragraph was cited from the shared `0Cx` family entry, validating the family fallback and focusing logic of §4.3. And the out-of-scope question produced the exact refusal with no citations, demonstrating that the grounding constraint holds rather than the model answering from general knowledge.

The evaluation set is small and was written by the developers; it demonstrates that the pipeline works end to end on representative cases, not that it generalises. Keyword scoring is a coarse proxy for correctness, which is why the full answers are retained for manual review.

8 Limitations and Future Work

8.1 Retrieval and Coverage

- **Corpus breadth.** Only System Messages Volume 8 (IEF–IGD) is indexed. Messages from other prefixes (for example IEC data-management messages, documented in other volumes) are not covered, and neither are user abends issued by application programs or Language Environment run-time messages.
- **Run-time evidence.** The assistant reasons over JCL configuration and documentation, not over the actual job output (SYSOUT, dumps, data set contents). It can explain what an S0C7 means for the program a step runs, but cannot identify the offending record.
- **Embedding model.** The small local embedding model and fixed ~700-character parts were chosen for cost and simplicity; a larger or domain-tuned embedding model and reranking were not evaluated.
- **Evaluation scale.** Ten scenarios with keyword scoring is a smoke test; a larger, independently written test set with graded human or model-judged correctness would be needed to measure accuracy.

8.2 Production Readiness

The authentication layer is sized for a multi-user proof of concept. The repository documents the following as separate work items before production use: HTTPS with a domain and certificate, followed by `JCL_COOKIE_SECURE=1` (the POC serves plain HTTP behind an IP allow list with test-only accounts); a managed, backed-up database in place of a single SQLite file; workspaces and roles; company single sign-on (SAML or OIDC); invitations, sign-up and an administration screen; password recovery and two-factor authentication; server-side chat history; an audit log of sign-ins and questions; configurable password and session policy; and lockout that cannot be used to deny service to a colleague — currently anyone who can reach the application can lock an account for 15 minutes — via per-IP or device-aware throttling or CAPTCHA.

9 Conclusion

The JCL Analyzer & Abend Assistant shows that a retrieval-augmented assistant can give mainframe engineers short, actionable explanations of batch failures while keeping every claim traceable to IBM

documentation or to the job's own JCL. The central design decisions — structural chunking that preserves job, step, code and page identity; deterministic recognition and exact lookup in front of semantic search; a model constrained to numbered sources with an explicit refusal; and post-hoc citation validation — address the specific risk of fluent but unsupported answers in an operational setting. The system is deployed, reproducibly built and continuously tested, and passed its first end-to-end evaluation. Broadening the corpus, incorporating run-time job output and hardening the authentication layer are the principal steps toward production use.

References

1. Amazon Web Services. *CardDemo: AWS Mainframe Modernization sample application.* github.com/aws-samples/aws-mainframe-modernization-carddemo, commit 59cc6c2.
2. IBM Corporation. *z/OS 3.1 MVS System Codes* (ieah700_v3r1.pdf). ibm.com/docs/en/SSLTBW_3.1.0.
3. IBM Corporation. *z/OS 3.1 MVS System Messages, Volume 8 (IEF–IGD)* (ieam800_v3r1.pdf). ibm.com/docs/en/SSLTBW_3.1.0.
4. Anthropic. *Claude Sonnet 4.5*, accessed via Amazon Bedrock (model ID `us.anthropic.claude-sonnet-4-5-20250929-v1:0`).
5. Reimers, N. and Gurevych, I. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *EMNLP*, 2019. (all-MiniLM-L6-v2 model, sentence-transformers.)
6. Lewis, P. et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *NeurIPS*, 2020.
7. OWASP Foundation. *Password Storage Cheat Sheet* (scrypt parameters).

Appendix A Configuration Reference

Setting	Default	Purpose
`BEDROCK_MODEL_ID`	`us.anthropic.claude-sonnet-4-5-20250929-v1:0`	Model used for generation
`AWS_REGION`	`us-east-1`	Bedrock region
`JCL_ASSISTANT_STUB`	`unset`	`1` returns a fixed answer (no AWS, no index)
`JCL_DB`	`backend/jcl.db`	SQLite file with accounts and sessions
`JCL_COOKIE_SECURE`	`0`	`1` adds the Secure cookie flag (HTTPS only)
`IBM_PDFS_S3`	`unset`	S3 prefix holding the IBM PDFs (setup.sh)
`REBUILD_INDEX`	`0`	`1` forces a corpus/index rebuild (setup.sh)

Table A1. Environment variables.

Constant	Value	Location and effect
`MAX_TURNS`	4	`rag/answer.py` — model calls per question
`ENTRY_CHARS / SNIPPET_CHARS`	6000 / 400	`rag/answer.py` — source text sent to the model / returned to the client
`HISTORY_TURNS / HISTORY_CHARS`	3 / 2000	`rag/answer.py` — conversation context
`PART_CHARS`	700	`ingest/index.py` — maximum embedded part size
`MAX_JOBS`	2	`rag/jobs.py` — jobs matched per question
`SESSION / REMEMBER`	12 h / 30 d	`api/auth.py` — session lifetime
`MAX_FAILED / LOCK`	5 / 15 min	`api/auth.py` — lockout policy

Table A2. Tunable constants in the source.